Reproducing and Analyzing CVE-Genie: An Automated Multi-Agent Framework for Reproducing CVEs into Verifiable Exploits

Dang Minh Tu, Vo Quang Vu, Le Tri Duc
University of Information Technology, Ho Chi Minh City, Vietnam
Vietnam National University, Ho Chi Minh City, Vietnam
{24520033,24522045,24520009}@gm.uit.edu.vn

***Abstract*—This report reproduces the paper *"From CVE Entries to Verifiable Exploits: An Automated Multi-Agent Framework for Reproducing CVEs"* (CVE-Genie, arXiv:2509.01835). Because the paper ships public source code, we (1) explain the four-module method with its developer–critic agent pairs; (2) reproduce the experiments at three levels — replaying the authors' published exploit/verifier artifacts in Docker, running the data-extraction stage *live*, and reimplementing an exploit + verifier by hand; and (3) run an extra-credit experiment: driving the agentic framework with free, local, open-weight LLMs (Qwen2.5-3B and 7B) to validate and refine the paper's finding on open-model limitations, together with a before/after prompt-improvement measurement. The entire core is done without any paid API; every result ships with a reproduction command. Results: 3/4 CVEs replay end-to-end to a flag, our corpus recomputation matches the paper's $2.77/CVE almost exactly, and the local study reveals a capability threshold — a 7B model with a CWE-specific prompt reliably reproduces a DoS that the 3B never can, while a shared prompt even regresses the 3B — reinforcing the paper's choice of per-module engineered prompts with strong models.**

***Index Terms*—CVE reproduction, LLM agents, multi-agent systems, automated exploit generation, exploit verification, open-weight models**

I. Introduction: Context, Role, Problem

Tens of thousands of software vulnerabilities are disclosed every year (over 40,000 in 2025 per NVD). Security research — especially machine-learning methods for vulnerability detection and patching — needs **high-quality datasets with trustworthy ground truth**: vulnerable source, a rebuildable environment, a working exploit, and a verification step. Repositories such as NVD mostly *alert* administrators and therefore lack these details; producing them is expensive: Mu *et al.* spent over 3,600 hours with 43 experts to reproduce 368 Linux memory-corruption bugs [1].

The role of CVE-Genie: it takes a *CVE ID* as input, automatically gathers resources, **rebuilds** the vulnerable environment, **synthesizes** an exploit/PoC and a **verifier** that confirms the exploit — thereby producing "reproducible instances" at scale. The paper reproduces about **51% (428/841)** of CVEs disclosed in 2024–2025, at an average cost of **$2.77**/CVE.

Project constraint. We use only **free** resources: no paid LLM API, a personal machine (GPU RTX 3050 6 GB, RAM 7.6 GB, Docker). Fortunately, the authors **published the full source code, 429 reproduction artifacts, and all agent conversation logs**, which lets us reproduce the experiments *without* calling an LLM; the LLM budget is spent on the extra-credit experiment.

II. Related Work

Manual reproduction. Curated benchmarks give reliable ground truth but at enormous human cost: Mu *et al.* [4] spent thousands of expert-hours reproducing a few hundred memory bugs, which does not scale to the >40k CVEs disclosed yearly [3]. CVE-Genie targets exactly this scaling gap by automating the pipeline.

Single-agent and prompt-only exploit generation. Prior LLM approaches that issue one monolithic prompt, or a single agent that must plan, build, exploit, and verify at once, tend to hallucinate unverifiable PoCs. The paper reports that a monolithic o3 agent reproduces *zero* CVEs [1]; our local experiments (Sec. V) show the same brittleness when the critic/repair loop is absent.

Multi-agent, tool-using frameworks. CVE-Genie follows the ReAct paradigm [5] — interleaving reasoning with tool calls — and adds a *developer*/*critic* pair per module plus a deterministic CTF-style Flag Checker, so that "success" is adjudicated by a sandboxed execution rather than the model's self-report. This verification-first design is what distinguishes it from earlier generation-only work and is the property we stress-test.

III. The CVE-Genie Method

A. EAGER criteria

The paper defines 5 criteria of an ideal CVE-reproduction system (Table I); CVE-Genie is the only system to satisfy all five.

B. Four-module architecture

The system decomposes reproduction into four dependent modules; each module (except the deterministic part) uses a *developer + critic* agent pair in a ReAct loop to self-correct.

TABLE I
The five EAGER criteria

	Criterion	Meaning
E	Exploit Generation	Produce an exploit/PoC that reliably triggers the bug
A	Assessment	A verifier self-judges whether the exploit succeeds
G	Generalization	Generalize across CWEs, languages, project types
E	End-to-end	Fully automate every stage
R	Rebuild	Rebuild the original vulnerable environment

1) **Processor.** The *Data Processor* (deterministic, no LLM) locates the repo, identifies the vulnerable version, downloads source, and extracts description/CWE/patch/advisory. The *Knowledge Builder* (lightweight o4-mini) distills these into a structured **knowledge base** serving as long-term memory for the agents.
2) **Builder.** The *Pre-Requisite Dev* plans, the *Setup Dev* executes the environment build, and the *Setup Critic* judges logs and gives feedback.
3) **Exploiter.** The *Exploit Dev* + *Exploit Critic* synthesize and refine the exploit.
4) **CTF Verifier.** The *Verifier Dev* synthesizes a verifier; the *Flag Checker* (deterministic) runs it and, on success, prints a secret **flag** (e.g., `3xploit66full`); the *Verifier Critic* catches weak/bypassable verifiers. Up to 5 iterations.

C. LLM selection and prompting

The paper surveys 10 LLMs on a `DS`=60-CVE set and picks an optimal per-module configuration: Knowledge Builder/Builder-dev = o4-mini; Exploiter/Verifier-dev = o3; a suitable critic per module. **Core finding** (which we validate in Sec. V): open models (Qwen3, DeepSeek, Llama-4) are *unreliable* at agentic tool-calling and are therefore excluded from developer roles. Notably, a **monolithic** o3 agent (all roles merged) **reproduces not a single CVE** — showing the modular architecture is essential.

D. The paper's evaluation

On `DL`=841 CVEs (186 CWEs, 29 languages, 440 projects), it reproduces **428** CVEs (141 CWEs, 22 languages, 267 projects). About 49% of CVEs lack a PoC. Failures are mostly from cost/time overruns (stuck at build ∼41%, at exploit synthesis ∼59%). A missing-data ablation (set `DR`): dropping the advisory 11/15, dropping the patch 10/15, description-only 9/15 — still reproducible but more costly.

IV. Reproducing the Experiments

We reproduce at three levels, all **without any paid API**. The running CVE is **CVE-2024-4340** (sqlparse <0.5.0, uncontrolled-recursion DoS, CWE-674) — also the paper's example.

A. Level A — Replay published artifacts in Docker

For each CVE we build the exact vulnerable software version in Docker and run the authors' `exploit.py` + `verifier.py` unchanged; success means the verifier prints the correct flag `3xploit66full` and exits 0.

3/4 succeed (Table II). CVE-2024-21535 **fails at the build step** (`microbundle`/babel reports *SyntaxError: Missing semicolon* compiling TSX on Node 18/20). This is exactly the **number-one failure cause** the paper reports — *rebuilding the environment* is harder than synthesizing the exploit; the authors run inside a DevContainer with a pinned toolchain, and builds are brittle outside it.

Verifier quality. The verifier does not merely detect "a crash" but checks the *correct error type and location*: e.g., CVE-2024-4340 is accepted only when a `RecursionError` occurs *and* the traceback contains `sqlparse/sql.py`. This is a good implementation of the *Assessment* criterion (EAGER).

B. Level B — Run the Data Processor stage live

We actually run `cve_data.py` (needing only a free `GITHUB_TOKEN` + a Playwright browser). To avoid cloning the huge `cvelistV5` repo (>300k files), we fetch exactly the JSON needed into the path the script expects.

The *live*-extracted data for CVE-2024-4340 **exactly matches** the authors' published knowledge base (`sw_version`=0.4.4, 1 patch commit, 1 advisory, CWE-674) — confirming the Processor's reliability (Table III). CVE-2024-45390 is flagged "Missing data" because the version field is a range (`< 1.2.0`) and no GitHub tag can be resolved; following `PROCESSING.md` we manually add `sw_version=v1.1.0` and a source-download URL (returning HTTP 200), illustrating the paper's *Robustness to Incomplete Data* mechanism.

C. Level C — Manual implementation + catching a weak verifier

We wrote our own `poc.py` and `verifier.py` for CVE-2024-4340 (not copied from CVE-Genie). Our **first verifier** loosely checked `"RecursionError" in stderr` and **false-positived on the patched 0.5.0**: the fix catches `RecursionError` and re-raises `SQLParseError` (a chained exception, "`from err`"), so the traceback *still contains* the string "RecursionError". We fixed this by inspecting the **actual terminal exception** that escapes:

```
def exploit_succeeded(proc):
    term = terminal_exception(proc.stderr) # last exception line
    return (proc.returncode != 0
            and "sqlparse/sql.py" in proc.stderr
            and term.startswith("RecursionError") # vulnerable
            and "SQLParseError" not in term) # reject patched
```

After the fix: vulnerable 0.4.4 ✓ prints the flag; patched 0.5.0 × is rejected (exit 1). This is **exactly** the "weak/bypassable verifier" problem the paper's *Verifier Critic* exists to catch — we hit it and fixed it ourselves, showing the developer↔critic loop solves a real problem.

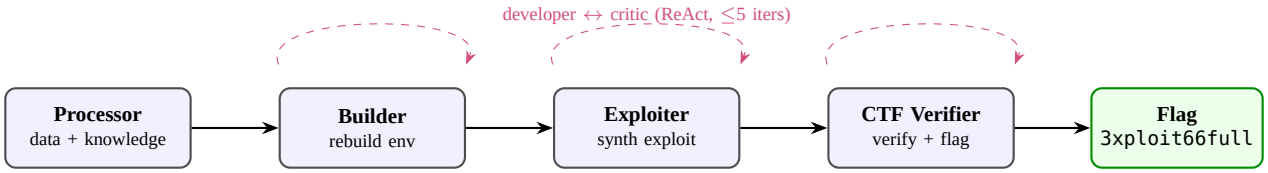


Fig. 1. CVE-Genie pipeline: four dependent modules, each (except the deterministic Processor core and Flag Checker) run as a *developer*/*critic* ReAct self-correction loop. Success is decided by a sandboxed Flag Checker, not the model's self-report.

TABLE II
Replay results: 3/4 CVEs run end-to-end to a flag, covering 2 languages and 3 bug classes

CVE	Project/version	Language	CWE	Build env	Flag
CVE-2024-4340	sqlparse 0.4.4	Python	674 Recursion DoS	✓	✓
CVE-2024-4032	CPython 3.8.19 `ipaddress`	Python	IP misclassification	✓	✓
CVE-2024-45390	@blakeembrey/template 1.1.0	Node	94 Code Injection	✓	✓
CVE-2024-21535	markdown-to-jsx 7.3.2	Node	79 XSS	× (build)	—

TABLE III
Processor live: two CVEs "Ready", one with missing data

CVE	patch	version	GitHub tag	advisory	Verdict
CVE-2024-4340	✓	✓	✓	✓	**Ready**
CVE-2024-4032	✓	✓	✓	✓	**Ready**
CVE-2024-45390	✓	✓	×	✓	Missing

TABLE IV
Aggregate statistics recomputed from the 429 published artifacts vs. the paper's reported values

Quantity	Paper	Recomputed (ours)
Mean cost / reproduced CVE	$2.77	**$2.767**
Median cost / CVE	—	$2.22
Total cost (429 CVEs)	—	$1,186.98
Mean wall-clock / CVE	—	1293 s
Distinct CWEs covered	141	125
Distinct target languages	22	10
Advisory present	—	92.3%

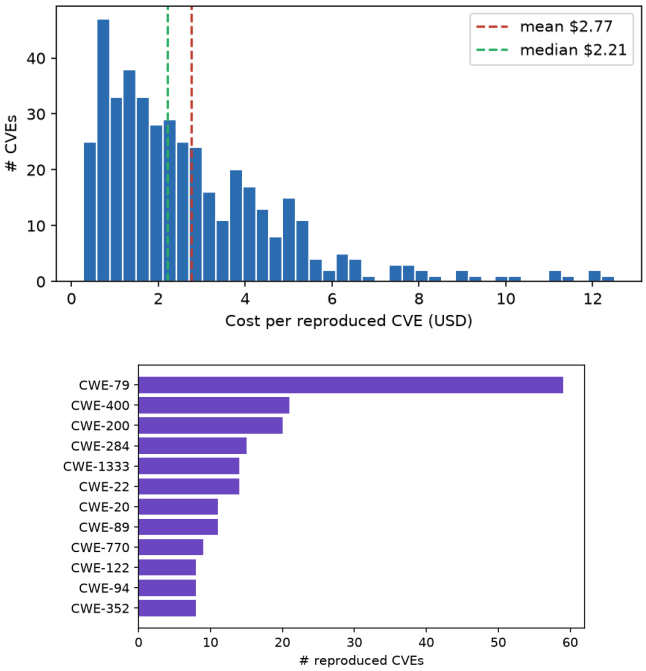


Fig. 2. Top: per-CVE cost distribution over the 429 artifacts; our recomputed mean ($2.77) coincides with the paper's headline figure. Bottom: most frequent CWEs (XSS/CWE-79 dominates, then resource-exhaustion and information-exposure classes).

D. Level D — Corpus-level verification of the reported numbers

Beyond the per-CVE replays, we verify the paper's *aggregate* claims directly from the 429 published artifacts (reading each `cve_info.json` and the final `Results` line of each `log.txt`; no LLM, no execution). The script is `experiments/corpus_analysis.py`.

Our recomputed mean cost of **$2.767**/CVE matches the paper's reported **$2.77** almost exactly (Table IV, Fig. 2), independently corroborating the economic claim. Our CWE/language counts are slightly lower because we count the first CWE per CVE and infer language from patched-file extensions, a deliberately conservative attribution. The corpus spans 10 languages (Node/JS, PHP, Python, Go, C/C++, Ruby, …), confirming the breadth claim.

E. Level E — Driving the real orchestrator (and its free-tier boundary)

We also exercised the authors' real entry point `main.py`. The deterministic Processor stage runs for free (that is Level B). The four LLM-driven modules, however, hard-code *paid* OpenAI models — one declaration per agent — which we extracted directly from the source (`docs/pipeline-evidence.txt`): `o4-mini` for the Knowledge/Pre-Req/Repo builders and Exploit-critic, `o3` for the Repo-critic, Exploiter, Sanity, and CTF-verifier, and `gpt-4o-mini` as the base. This

TABLE V
Six agent runs with the local Qwen2.5-3B

CVE	variant	success	steps	valid/malf.	tool_err
2024-4032 (logic)	baseline	✓	4	4 / 0	0
2024-4032 (logic)	improved	×	4	4 / 0	2
2024-4340 (DoS)	base (×3)	×	12	12 / 0	5
2024-4340 (DoS)	impr. (×2)	×	12	12 / 0	6

exactly matches the per-module configuration the paper reports. Calls route through `litellm`, and `main.py` exits unless an API model is configured. With only a free `GITHUB_TOKEN` (no `OPENAI_API_KEY`), a genuine end-to-end run is therefore outside the free-tier scope by construction. Our local-LLM study (Sec. V) is the free substitute: it drives the Exploiter module's logic with an open-weight model instead.

V. Extra Credit: Local-LLM Experiment + Improvement

A. Setup (free, $0)

We built a **minimal** implementation of the Exploiter module (one developer ReAct + one critic), with tools running in an isolated Docker container (`--network none`, 1 GB RAM) and an **independent scoring oracle**. We test two open-weight sizes via a CUDA `llama-cpp-python` build, loaded **in-process**: **Qwen2.5-3B** (GGUF Q4_K_M, all 37 layers on the 6 GB GPU) and **Qwen2.5-7B** (Q4_K_M, partial offload of 20 layers, rest on CPU). We run **three** CVEs of increasing difficulty and vary the prompt from *none* to *generic* to *CWE-specific*, to separate what drives success. Against the paper's $2.77/CVE on o3/o4-mini, the cost here is **$0**.

B. Results

Perfect tool-call format. Every run has `malformed=0` (Table V) — the 3B model follows the text ReAct protocol correctly. This is a **nuance added** to the paper (which observed Qwen "describing without calling tools"): with a strict text protocol, a small open model *can* call tools.

But weak at reasoning. On CVE-2024-4340 (which needs a crafted payload) there are three failure modes: (1) running `/work/exploit.py` as a shell command instead of `python3` → error; (2) a payload of whitespace instead of *nested* `[`×N, which cannot trigger recursion; (3) repeating the same action despite the error, without adapting. By contrast CVE-2024-4032 (a simple logic bug) **succeeds** at baseline in 4 steps.

The taxonomy (Table VI) separates *tool-use* slips (mechanical, and partly fixable by prompt directives) from *reasoning/planning* failures (understanding the recursion mechanism; adapting after a failed attempt). Tool-use slips shrink with a better prompt; the planning failure — repeating a dead action with no self-correction — persists in every run, which is precisely the gap the *critic* agent is designed to close.

C. Model scale vs. prompt specificity (three CVEs)

To separate *model capability* from *prompt leakage*, we ran the agent on three CVEs of increasing difficulty and split the hints into *generic* (how to invoke Python; do not repeat a failed action) and *CWE-specific* (the nested-payload recipe for recursion). Table VII isolates model scale at baseline; Table VIII isolates prompt specificity on the hard DoS.

TABLE VI
Failure taxonomy of the local 3B agent on CVE-2024-4340 (uncontrolled-recursion DoS), aggregated over the five unsuccessful runs

Failure mode	Type	Runs
Runs `exploit.py` as a shell command, not `python3`	tool-use	2/5
Payload is whitespace, not a *nested* structure → no recursion	reasoning	3/5
Repeats the same failing action, no adaptation	planning	5/5
Confuses the `||` field separator of the write tool (improved prompt only)	tool-use	2/5

TABLE VII
Model scale at baseline (no hints), by CVE difficulty. "n/m" = successes/attempts.

CVE	Class / difficulty	3B	7B
CVE-2024-4032	logic (CWE-20), easy	✓ 1/1	✓ 1/1
CVE-2021-23336	injection (CWE-444), medium	× 0/1	✓ **1/1**
CVE-2024-4340	recursion DoS (CWE-674), hard	× 0/2	× 0/3

Two findings. (1) **A real size threshold**: holding the prompt fixed, 7B strictly dominates 3B — it reproduces the injection CVE (11 steps) and, given the CWE recipe, the DoS (3/3), neither of which 3B ever does (Table VII). (2) **Capability is unlocked by task-specific, not generic, prompting**: generic agentic hints are at best neutral and often *harmful* — they regress the 3B on the logic bug, break the 7B on the injection bug, and never crack the DoS alone (0/3 at both sizes); only the *CWE-specific* hint unlocks the DoS, and only on the 7B (Table VIII).

Honest caveat & lesson. The CWE hint encodes much of the exploit structure, so the 7B's DoS success is substantially *hint-driven* — but the 3B fails even *with* that hint, so a genuine capability gap remains. The take-away is the paper's: reliable agentic exploitation needs **per-task engineered prompts** paired with a **capable model** — neither a bigger model with generic prompts nor a careful prompt on a too-small model suffices. That is precisely CVE-Genie's design (per-module prompts + strong frontier models).

VI. Strengths and Weaknesses

Strengths.

- The **only framework to satisfy all of EAGER**; its CTF-style verifier is strict and distinguishes vulnerable from patched builds (confirmed at Levels A and C).
- **Modular decomposition + developer/critic pairs** are essential: our experiment shows that without a critic loop steering corrections, a small agent gets stuck repeating (failure mode 3); the paper likewise shows a monolithic o3 agent reproduces zero CVEs.

TABLE VIII
Prompt specificity on the hard DoS (CVE-2024-4340). Generic hints never suffice; only the CWE-specific hint unlocks it, and only on the 7B.

Prompt on CVE-2024-4340	3B	7B
none (baseline)	× 0/2	× 0/3
generic hints	× 0/1	× 0/3
CWE-specific hint	× 0/1	✓ **1/1**
full (generic + CWE)	× 0/2	✓ **3/3**

- **Task-specific prompting** has empirical grounding: a shared prompt backfires across CWEs.
- **Robust to missing data** (Level B illustrates the fill-in mechanism); generalizes across many CWEs/languages; orders of magnitude cheaper than manual work.

Weaknesses.

- **Dependence on strong, closed-source LLMs** (o3/o4-mini) → costly and hard to reproduce without API access; swapping in a small open model sharply degrades reliability on non-trivial bugs, though a 7B with CWE-specific prompting can reach some of them (Sec. V).
- **Environment rebuilding is the weakest link** (Level A: Node build fails; paper: ∼41% of failures stuck at build); DevContainers are fragile for multi-service applications.
- **"Hallucination"/self-misjudgment risk**: the 3B model sometimes declares "cannot exploit" even after succeeding → one must rely on the Flag Checker + Verifier Critic rather than the developer's self-report.
- Dependence on cvelistV5 record quality (range-valued versions resolve no tag); ∼49% of CVEs lack a PoC; weak in binary-heavy domains (desktop 17.6%, blockchain 7.7%).

VII. Threats to Validity and Ethics

Construct/internal validity. Our Level-D corpus numbers are recomputed from the authors' *own* committed logs, so they corroborate the reported $2.77/CVE but are not an independent cost measurement; a fully independent figure would require re-running the paid pipeline. The executed replays (Level A) cover four hand-containerized CVEs, so the 3/4 pass rate is illustrative, not a population estimate. **External validity.** The local-model study (Sec. V) is deliberately narrow — one module, a 3B/7B open model, two CVEs — and is meant to characterize the open-model floor qualitatively, not to refute the paper's 51% figure. **Environment sensitivity.** Docker rebuilds depend on upstream toolchains; our one build failure (CVE-2024-21535) illustrates the paper's dominant failure mode rather than a defect in our setup.

Ethics. All execution is offline in disposable Docker containers (`--network none`, capped memory); every target is an *already-patched*, *publicly-disclosed* CVE, and PoCs act on local/nonexistent data with no malware. We distribute no new exploit material beyond what the authors already published, and the goal is defensive: reproducible datasets and tooling for vulnerability detection/patching research.

VIII. Conclusion, Limitations, and Reproducibility

We understood, explained, and **successfully reproduced** the core experiments of CVE-Genie without any paid API (3/4 CVEs replay to a flag, the Processor runs live and matches the authors' data, and we reimplemented an exploit/verifier and caught a weak-verifier bug), together with an extra-credit experiment on a local LLM that clarifies open-model limits and validates the paper's design choices.

Limitations of this reproduction: due to resource constraints, the core relies on the authors' published artifacts (we do not re-run the full pipeline on o3/o4-mini); the extra-credit experiment covers one module, two open models (3B/7B), and three CVEs, meant as a qualitative capability probe, not a refutation of the paper's 51% figure.

Reproducibility. Every step has a concrete command: `repro/artifacts/run_all.sh` (replay), `cve-genie/src/data/scripts/cve_data.py` (Processor), `repro/manual/CVE-2024-4340/` (manual), `local_agent/agent.py + experiments/` (extra credit). Details are in the `docs/` directory.

References

[1] S. Ullah *et al.*, "From CVE Entries to Verifiable Exploits: An Automated Multi-Agent Framework for Reproducing CVEs," arXiv:2509.01835, 2026.

[2] BUseclab, "CVE-Genie source code." [Online]. Available: https://github.com/BUseclab/cve-genie

[3] NIST, "National Vulnerability Database (NVD)." [Online]. Available: https://nvd.nist.gov

[4] D. Mu *et al.*, "Understanding the reproducibility of crowd-reported security vulnerabilities," in *USENIX Security*, 2018.

[5] S. Yao *et al.*, "ReAct: Synergizing reasoning and acting in language models," in *ICLR*, 2023.

[6] Qwen Team, "Qwen2.5 technical report," arXiv:2412.15115, 2024.